SC2006 Lab 3 — AI-based Project Initialisation (§3.3)

Team 5 · GoWhere · AI used: ChatGPT (free tier, web interface; the model name was not recorded) · Date: 29 October 2026

Both prompts were run in fresh chat sessions. The AI's full responses are reproduced word for word in Appendices A and B. The skeleton it produced is submitted as `lab3/ai_report/ai_generated_skeleton.zip`, kept apart from our own application skeleton in `gowhere/`.

Part 1 — Technology stack re-prompt (§3.3.1, §3.3.2)

1.1 What the AI was given

We sent the **same prompt** as in Lab 2, word for word. We could not reuse the Lab 2 chat session itself, so this was a new session. What changed is the material we uploaded with the prompt:

	Lab 2 (previous exercise)	**Lab 3 (this exercise)**
Application	*Neighbourhood Comparison App*: one feature	*GoWhere*: three features — Where to Live, Where to Lepak, Where to Eat
Uploaded material	The Lab 1 deliverable: FR, NFR, Data Dictionary, use case diagram and descriptions, UI mock-ups	The current requirements document (FR, NFR, Data Dictionary, use case model UC-1 to UC-13) **and the current design class diagram** (40 classes, as Mermaid source)
Class diagram uploaded?	No. Our Lab 2 report lists the uploaded material, and it contains no class diagram	Yes. Classes marked boundary / control / entity, one adapter per External Service, and the Strategy hierarchy for scoring
Scope described	Compare 2–4 Planning Areas on five factors, using government data only	Six switchable factors; events extracted from Telegram by an LLM; food places; live carpark data; Google Places ratings

1.2 Prompt

> Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team

1.3 Response, summarised (full text: Appendix A)

Area	**Lab 2 recommendation**	**Lab 3 recommendation**	
Language, framework	Python, Flask	Python, Flask (Django considered and rejected: no accounts needed)	Same
Front end	HTML, CSS, JavaScript	HTML, CSS, vanilla JavaScript, Jinja; React explicitly avoided	Same
Charts	Chart.js	Chart.js	Same
Testing	pytest	pytest, plus pytest-cov, `unittest.mock`, Playwright later	Expanded

Maps	Leaflet.js	"OneMap API"	Changed
Data processing	Pandas, GeoPandas, Shapely	pandas, NumPy — **no geospatial library**	Changed
Storage	"Local versioned data snapshot", JSON/CSV	SQLite in three separate stores, an in-memory session store, ratings never stored	Changed
External services	None named	`requests`, with **one adapter per service**: OneMap, LTA DataMall, Telegram, Google Places, LLM	New
Scheduling	A periodic refresh pipeline	Cron / scheduled jobs: ingestion every 6 h, maintainer-triggered rebuilds	Expanded
Tooling, deployment, security	Git, GitHub	Adds Ruff/Black, GitHub Actions, virtual environments, a hosting choice, a security-controls table and a list of technologies to avoid	New

1.4 How similar are the recommendations? (§3.3.2)

Mostly the same at the core, much larger around it. The core is unchanged: the language, the web framework, the front-end approach, the chart library and the test framework. What changed is data storage, maps and geospatial processing. Everything about external services, operations and security is new.

§3.3.2 (1) — what caused the differences. Our prompt and our language preference were the same both times, so the differences come from the inputs. Two inputs changed, and the response lets us tell their effects apart, because ChatGPT marks which uploaded file each statement relies on.

Caused by the class diagram (the Lab 2 input had none, so these had nothing to anchor on before):

- **"Modular monolith"** — the response justifies this by the diagram's existing boundary / control / entity split.
- **The adapter rule** ("do not scatter direct API calls") — taken from `OneMapAdapter`, `DataMallAdapter`, `LlmAdapter` and the others. The response even sketches `OneMapAdapter`.
- **SQLite, split into three stores** — mirrors the diagram's three entity stores, `Snapshot`, `EventStore` and `PlaceStore`. Lab 2's flat "JSON/CSV snapshot" had no such structure to follow.
- **The in-memory session caveat** ("do not rely on an in-memory session store across multiple workers") — read from `SessionStore` / `SessionState`. This is the same reasoning behind our own decision to run as one process.
- **Dropping GeoPandas/Shapely, and "OneMap" for maps** (our inference; the response does not say why). The diagram shows only the classes the running app uses. Geometry arrives already computed in `Snapshot`, so no class processes geometry, which most likely explains why the geospatial library was dropped. No class draws a map either; the only map-related class is `OneMapAdapter`, which most likely explains why OneMap was named as the map technology. This tells us something about *our* diagram: it leaves out the offline data build. Part 2 shows the same gap again.

Caused by the requirements document (these are cited to it):

- The new External Services (Telegram, LLM, Google Places, DataMall) and the Scheduling rows. Lab 1 had none of these features.
- The privacy controls: session-only postal codes, ratings never stored, and the warning that cookie-based session storage may break the Security NFR.

- The data-freshness rules: carpark data at most two minutes old; keep the last good snapshot if a refresh fails.

§3.3.2 (2) — not applicable. We have not changed programming language or framework since Lab 2. Both labs use Python and Flask.

How the advice compares with what we built. We agree with most of it, and the shape we built matches: Flask, Jinja, vanilla JavaScript, Chart.js, SQLite, `requests`, pytest, an adapter per service, and a single process with in-memory sessions. We differ in three places:

- **pandas/NumPy** — we don't use either. Scoring is medians and percentiles over about 10,800 HDB blocks, which plain Python handles. That is one fewer dependency for teammates to install.
- **Maps** — OneMap supplies data (search, routing), but drawing a map in a browser needs a JavaScript map library. We use Leaflet (Lab 2's answer) and MapLibre. On this point the Lab 2 answer was the better one.
- **Serving** — "a Flask application process" is incomplete. Flask's built-in server is meant for development only, so we serve the app with Waitress.

Ruff and GitHub Actions are sensible suggestions that we have not adopted yet.

Part 2 — AI-generated minimal skeleton (§3.3.3, §3.3.4)

2.1 Prompt

New chat session. We uploaded **only** the class diagram, as the manual specifies ("using your class diagram as input"):

> The attached file is the design class diagram of a web application, written in Mermaid.
>
> Generate a minimal skeleton template for this project in Python 3, using Flask.
>
> Rules for the files:
> 1. Create a folder hierarchy containing all the files you think the project needs (source, templates, tests, configuration).
> 2. Every file must be empty except for TODO comments describing what will go in it.
> 3. Do not write any code at all. No imports, no class or function definitions, no method signatures, no variables, no configuration values, no HTML tags. Only comments.
>
> Package the whole folder hierarchy as a single .zip file I can download.
>
> After the zip, show the folder tree, and give one sentence per folder explaining why it exists.

No follow-up prompt was needed: the zip came with the first response.

2.2 What it produced (full text: Appendix B)

- 74 files under `app/boundary/`, `app/control/` and `app/entity/` (one folder per stereotype), plus `templates/` grouped by feature, `static/`, `tests/` and `config/`.
- **Every file is exactly one line: a single TODO comment.** We checked this with a script over the extracted zip: 74 files, 74 lines in total.

2.3 Critical evaluation (§3.3.4)

To what extent do we agree with its choices?

- **Packages follow the diagram's stereotypes** (boundary / control / entity). This is exactly what manual §3.4.2 suggests, and it is a faithful reading of our diagram.
    - Our own `gowhere/` is organised differently, **by feature**: `scoring/`, `lepak/`, `eat/`, `adapters/`, `services/`, `ingest/`, `etl/`, `web/`. The stereotypes are shown on the class diagram, not as folders. We chose this so that a change to Where to Eat stays in one folder (high cohesion) and two developers working on different features rarely edit the same folder.
    - Both arrangements are defensible. The AI's matches the diagram more closely; ours matches how the work was divided.
- **One file per scoring strategy**, next to `scoring_strategy.py` and `scoring_engine.py`, makes the Strategy pattern visible in the folder tree. We agree.
- **External integrations kept at the boundary**, `.env.example` for secrets, and the application factory named in `app/__init__.py`: all agree with our design. We have a `create_app()` factory.
- **The TODOs say too little.** The diagram gives `ScoringStrategy` nine operations, including `category_scores`, `precompute` and `validate_options`. The skeleton's whole file is "*TODO: Document the shared scoring-strategy contract…*".
    - Our rules banned signatures, but not a comment *listing* the operations still to write. The AI chose to say less than it was allowed to, and most of the diagram's detail did not reach the skeleton.
    - Every TODO also begins "*Document …*", which reads as writing documentation rather than implementing anything.

Has it missed something obvious, or failed to produce anything reasonable?

1. **No home page.** There are templates for each feature's pages, but none for the screen the dialog map starts from: the page that leads to the three features.
2. **No offline data build.** `data_gov_adapter.py` and `overpass_adapter.py` exist, but nothing in the skeleton calls them, and nothing builds the neighbourhood snapshot or the place list (UC-9, UC-13).
    - The cause is ours: our class diagram shows these two adapters but none of the build classes that use them. It is the same gap seen in §1.4.
    - **Action:** add the offline build to the class diagram.
3. **No `data/` folder** for the databases and the hand-kept reference lists. The Part 1 response, given the requirements, did include one.
4. **Only a development entry point.** `run.py` is described as being for development. Nothing serves the app in production or runs ingestion on its background schedule. Our `serve.py` does both.
5. **Uneven test files.** Six scorer files but a single scoring test file, and one test file for all seven adapters.
6. **No place for the map and chart libraries** in `static/`. We serve Leaflet, MapLibre and Chart.js from our own server, never from a CDN, so the app still works offline. That is an architecture decision the class diagram doesn't show, so this one is fair to the AI.

Overall, it did produce something reasonable: a complete, consistently named tree that mirrors the diagram one-to-one.

Did it add anything beyond what was requested?

- **No code.** All 74 files are comment-only. The HTML templates use HTML comment syntax (`<!-- TODO … -->`), which is a comment rather than a tag, so they also follow the "no HTML tags" rule.
- **Small extra files:**
  - Placeholder `TODO.md` files inside `static/css`, `static/js` and `static/images`. These are sensible, because a zip cannot hold empty folders.
  - A `pyproject.toml` alongside `requirements.txt`, which duplicates where dependencies would be declared.
- **Unrequested text in the response**: an "Architectural notes" section and a "Recommended next step". Both are harmless.
- **Separate modules for the `Event`, `Place` and `Neighbourhood` entities.** Our diagram marks these «entity, table» because our code carries their rows as plain dictionaries. Giving each its own module is a reasonable reading of a class diagram that doesn't know that.

Conclusion. The skeleton reproduces the class diagram faithfully and adds almost nothing to it. That made it a useful check on our design: everything missing from the skeleton — the offline data build, the start-up and serving code, the home page — is something our class diagram does not show either.

Appendix A — Part 1 response (tech-stack re-prompt), verbatim

Copied as ChatGPT displayed it. Its file-citation markers are shown as [cites: …]: `GoWhere_requirements_and_use_ca…` *is the requirements document,* `class_diagram_mermaid` *the class diagram.*

Recommended Technology Stack for GoWhere

1. Executive recommendation

Based on the GoWhere requirements document and class diagram, I recommend a **Python-based monolithic web application with server-rendered HTML, lightweight JavaScript, SQLite, and automated testing using pytest**.

For an entry-level software engineering team, the goal should be to choose technologies that are easy to learn, straightforward to maintain, and capable of meeting GoWhere's actual requirements without introducing unnecessary infrastructure.

GoWhere is a web application with three interconnected features:

- **Where to Live:** Compare 2–4 Singapore neighbourhoods using six weighted factors and recommend the best match.
- **Where to Lepak:** Find upcoming events based on categories, dates, travel time, and transport mode.
- **Where to Eat:** Find food establishments based on location, cuisine, opening hours, and travel preferences.

The application also integrates external services, including OneMap, LTA DataMall, Telegram, Google Places, and an LLM service. Its requirements emphasise reliability when external services fail, modularity, data accuracy, privacy, and response time. [cites: GoWhere_requirements_and_use_ca…]

My central recommendation is to keep the application as a modular monolith rather than split it into microservices. The class diagram already separates boundary adapters, control logic, and entities, providing a useful foundation for a small team to develop and maintain the system. [cites: class_diagram_mermaid]

2. Recommended stack at a glance

Layer	Recommended technology	Why it fits GoWhere
Programming language	Python	Suitable for application logic, data processing, scoring algorithms, and API integration.
Backend framework	Flask	Lightweight and approachable for a small team building a server-rendered web application.
Frontend	HTML5, CSS3, vanilla JavaScript	Supports interactive forms, filtering, maps, and charts without a complex frontend build system.

HTML templating	Jinja	Allows reusable page layouts and dynamic rendering of search results.
Data analysis	pandas, NumPy	Useful for neighbourhood statistics, weighted calculations, and data preparation.
Primary database	SQLite	Simple deployment and maintenance for a modest application with structured data.
Data visualisation	Chart.js	Suitable for the grouped bar charts required by the neighbourhood comparison feature.
Mapping	OneMap API	Provides Singapore-focused maps, location resolution, and route information already required by the design.
External API integration	Python `requests`	Keeps HTTP requests to external services simple and consistent.
Automated testing	pytest	Supports unit, integration, and regression testing of business logic and external-service adapters.
Version control	Git and GitHub	Supports collaboration, code review, issue tracking, and change history.
Code quality	Ruff and a formatter such as Black	Helps maintain consistent, readable Python code.
Dependency management	Python virtual environments and a pinned requirements file	Helps teammates reproduce the same development environment.
Deployment	A managed Python hosting service or a small Linux server	Avoids unnecessary infrastructure and operational complexity.
Automation	A scheduled job or cron	Supports periodic event ingestion and data-maintenance tasks.

This is a proposed stack, not a claim that every listed technology is already used by the implementation. The supplied design documents describe the system's behaviour and architecture but do not establish every technology choice in this table.

---

3. Backend: Python and Flask

Recommendation: Python + Flask

Python is a particularly suitable choice because GoWhere combines conventional web development with data processing, scoring, geographic calculations, and LLM integration.

The class diagram already identifies separate components for:

- Web request handling (`WebRoutes`)
- Search logic (`LepakSearch` and `EatSearch`)
- Neighbourhood scoring (`ScoringEngine` and `ScoringStrategy`)
- Location and routing (`LocationService`, `EventRouter`, and `CommuteRouter`)
- External services (`OneMapAdapter`, `DataMallAdapter`, and `LlmAdapter`)
- Data access (`Snapshot`, `EventStore`, and `PlaceStore`)

These responsibilities can be organised into Flask routes, Python service modules, data-access modules, and external-service adapters without requiring a large framework. [cites: class_diagram_mermaid, class_diagram_mermaid]

Why Flask rather than a more complex framework?

For an entry-level team, Flask offers several advantages:

1. **A small learning curve:** Developers can understand request handling, routing, and templates without learning a large framework first.
2. **Flexible architecture:** The team can retain the existing separation between boundary, control, and entity components.
3. **Straightforward integration:** Calling external APIs and processing their responses fits naturally into Python modules.
4. **Incremental development:** Each feature can be developed and tested separately while sharing common services.

Django would also be a reasonable choice, particularly if the application needed built-in authentication, an administrative interface, and a more extensive data-management layer. However, GoWhere explicitly does not require account creation, and its current design already has a relatively focused set of application responsibilities.

I would choose Flask unless the team has a specific requirement that Flask cannot meet conveniently.

Suggested project structure

```
gowhere/
├── app/
│   ├── routes/
│   │   ├── live.py
│   │   ├── lepak.py
│   │   └── eat.py
│   ├── services/
│   │   ├── scoring.py
│   │   ├── location.py
│   │   ├── routing.py
│   │   ├── parking.py
│   │   └── ingestion.py
│   ├── adapters/
│   │   ├── onemap.py
│   │   ├── datamall.py
│   │   ├── telegram.py
│   │   ├── google_places.py
│   │   └── llm.py
│   ├── repositories/
│   │   ├── events.py
│   │   └── neighbourhoods.py
│   ├── templates/
│   ├── static/
│   │   ├── css/
│   │   └── js/
│   └── __init__.py
├── data/
├── tests/
├── scripts/
├── requirements.txt
├── .env.example
└── README.md
```

This is a suggested organisation rather than a requirement to reproduce every existing class as a separate file. The important principle is that **routes should handle web requests, services should implement business rules, repositories should manage data access, and adapters should communicate with external services**.

---

4. Frontend: HTML, CSS, JavaScript, and Jinja

Recommendation: Server-rendered pages with lightweight JavaScript

I would start with:

- **HTML** for page structure and forms.
- **CSS** for layout, responsive design, and consistent styling.
- **Vanilla JavaScript** for interactive controls and asynchronous updates.
- **Jinja** for rendering Python data into HTML.
- **Chart.js** for the neighbourhood comparison chart.

I would not introduce React, Next.js, or a separate frontend application at the beginning.

Why this approach fits GoWhere

GoWhere has three distinct features, but most of their interactions are form-driven:

- Selecting neighbourhoods and configuring comparison weights.
- Selecting event categories, dates, travel modes, and maximum travel times.
- Filtering food establishments and sorting by distance, opening status, or rating.

These workflows can be implemented with server-rendered pages and targeted JavaScript enhancements.

This approach also supports the requirement that the application remain usable when certain presentation features are unavailable. For example, the requirements explicitly state that neighbourhood selection must remain usable when the map cannot be displayed. [cites: GoWhere_requirements_and_use_ca…]

A practical implementation would use:

Feature	Implementation
Search forms	HTML forms with server-side validation
Dynamic filters	JavaScript
Search progress indicators	JavaScript, with progress responses or asynchronous requests from the backend
Neighbourhood ranking	Server-rendered HTML
Grouped bar chart	Chart.js
Neighbourhood boundaries	OneMap-compatible mapping implementation using locally held boundary geometry
Event and food routes	OneMap API, with the resulting route details rendered in the interface
Responsive layouts	CSS media queries and flexible layouts

When would React become worthwhile?

Consider React if the application later requires extensive client-side state management, highly interactive map interfaces, or a much richer single-page experience.

For the current project, adding React would introduce another language ecosystem, a build process, frontend-backend integration concerns, and additional testing requirements.

Those costs are difficult to justify before the team has demonstrated that a simpler interface is insufficient.

---

5. Database and data management

Recommendation: SQLite initially

SQLite is a reasonable starting point because GoWhere maintains structured neighbourhood, event, and place data, while the class diagram already separates those data responsibilities.

The requirements also distinguish between relatively stable datasets and information that changes frequently. For example, neighbourhood data is refreshed through a dedicated procedure, while event ingestion is performed periodically and parking availability must be no more than two minutes old. [cites: GoWhere_requirements_and_use_ca…, GoWhere_requirements_and_use_ca…]

I would organise the data into three main groups.

Data group	Suggested storage	Reason
Neighbourhood snapshot	SQLite database	Structured data used by scoring and comparison logic
Event records	SQLite database	Supports event lookup, updates, deduplication, and expiry
Place list	SQLite database or a validated local dataset	Supports cuisine filtering, location searches, and place details
Session state	In-memory session store initially	Supports temporary user preferences and search state
API credentials	Environment variables or hosting-platform secrets	Keeps credentials outside the source code
Live parking data	Short-lived cache	Meets the two-minute freshness requirement
Place ratings	Retrieved on demand, not persisted	Respects the requirement not to store rating data

A key design decision: separate static and dynamic data

I would use SQLite as the default database for structured operational data, but I would not force every dataset into the same update process.

Neighbourhood data: Build a new snapshot, validate it, and only then make it active. If validation fails, continue using the last successful snapshot.

Events: Store validated event records, track ingestion progress, and remove ended events according to the requirements.

Places: Regenerate and validate the place list separately from event ingestion.

Live parking: Cache responses briefly and refresh them when necessary.

The requirement to preserve the previous valid neighbourhood snapshot is especially important: a failed data refresh must not make the live application unusable. [cites: GoWhere_requirements_and_use_ca…]

When should the team migrate to PostgreSQL?

Consider PostgreSQL if GoWhere develops more demanding concurrent writes, requires a shared database across multiple application instances, or needs more sophisticated relational queries and operational controls.

SQLite's simplicity is valuable during early development, but it should not be treated as the automatic long-term answer for every deployment scenario.

One additional consideration is geographic querying. If the project eventually needs extensive spatial queries directly in the database, the team could evaluate PostgreSQL with PostGIS. However, I would not add that complexity unless the workload demonstrates a need for it.

---

6. Data processing, maps, and external APIs

GoWhere's external integrations are central to its functionality, so I would prioritise reliability and clear interfaces over adding more infrastructure.

Recommended libraries and services

Requirement	Recommendation	Implementation approach
Neighbourhood calculations	pandas and NumPy	Process structured data and implement scoring calculations
Maps and routing	OneMap API	Resolve Singapore locations and retrieve routes
Parking availability	LTA DataMall	Retrieve current carpark availability
Event collection	Telegram integration	Fetch posts from configured public channels
Event extraction	An LLM API through `LlmAdapter`	Convert post content into structured event records
Place ratings	Google Places API	Retrieve ratings on demand within the configured daily cap
HTTP communication	Python `requests`	Centralise request handling, timeouts, and response parsing

The documents identify OneMap, DataMall, Telegram, Google Places, and the LLM service as external dependencies. They also specify that each external service should be accessed through its own adapter with a documented fallback. [cites: GoWhere_requirements_and_use_ca…]

The most important architectural rule: use adapters

Do not scatter direct API calls throughout the application.

Instead, make each adapter responsible for communicating with one external service.

For example:

```
class OneMapAdapter:
    def search(self, query):
        """Resolve a location using OneMap."""
        ...

    def route(self, start, end, route_type):
        """Retrieve route information."""
        ...
```

The implementation above is illustrative, not a complete working API client.

The calling service should depend on the adapter's interface rather than on the details of HTTP requests, authentication, and response formats.

This allows the team to:

- Replace an API provider without rewriting business logic.
- Test the application without making real API calls.
- Handle timeouts and failures consistently.
- Apply caching and rate limits in one place.
- Mock unavailable services during development.

This design is already reflected in the class diagram's boundary adapters and shared location and routing services. [cites: class_diagram_mermaid, class_diagram_mermaid]

External-service reliability

Each adapter should have:

1. A finite request timeout.
2. Appropriate handling for HTTP errors and malformed responses.
3. Validation of returned data.
4. A documented fallback or graceful degradation strategy.
5. Logging that records useful operational information without exposing secrets or private user inputs.
6. Unit tests using mocked responses.

For example, if OneMap cannot calculate a route, the event search should still display the event with a clearly labelled straight-line distance estimate where the requirements permit it.

If Google Places ratings are unavailable, food results should still be displayed without ratings, and rating-based sorting should be disabled.

These are specified behaviours, not merely optional enhancements. [cites: GoWhere_requirements_and_use_ca…, GoWhere_requirements_and_use_ca…]

---

7. Testing and code quality

For an entry-level team, I would prioritise a strong automated testing workflow over adding advanced infrastructure.

Recommended testing stack

- **pytest:** Main test framework.
- **pytest-cov:** Coverage measurement.
- **unittest.mock:** Mock external APIs and isolate components.
- **Ruff:** Python linting and code-quality checks.
- **Black:** Optional code formatter if the team prefers it.
- **GitHub Actions:** Automatically run tests when code is pushed or a pull request is opened.

Three levels of testing

Level 1: Unit tests

Test individual components in isolation.

Examples:

- Does the scoring engine calculate weighted means correctly?
- Does the housing-affordability scorer enforce the minimum transaction threshold?
- Does the opening-hours parser handle a closed venue correctly?
- Does the search logic exclude events beyond the configured travel-time limit?

The scoring engine is particularly suitable for unit testing because the requirements specify exact calculation formulas and rounding behaviour. [cites: GoWhere_requirements_and_use_ca…]

Level 2: Integration tests

Test how components work together.

Examples:

- Can the event repository store and retrieve a valid event?
- Does the search service handle a mocked OneMap response?
- Does a failed data refresh leave the existing snapshot active?
- Does event ingestion preserve unprocessed posts when a dependent service fails?

Level 3: End-to-end tests

Test complete user workflows through the browser.

Examples:

- Compare neighbourhoods and view the recommendation.
- Search for an upcoming event and open its route.

- Search for food establishments and filter the results.

A browser automation tool such as Playwright can be added once the main workflows are stable.

What should be tested first?

I would prioritise:

1. Scoring calculations and ranking.
2. Input validation.
3. External API failure handling.
4. Data snapshot replacement.
5. Session isolation and privacy.
6. Search sorting and filtering.
7. End-to-end user workflows.

The requirements explicitly cover concurrent requests, external-service failures, data freshness, and session-only handling of location and other user inputs. These need dedicated tests rather than relying on a successful manual demonstration. [cites: GoWhere_requirements_and_use_ca…]

---

8. Git, GitHub, and team collaboration

For a beginner team, I recommend using GitHub with a lightweight pull-request workflow.

Suggested workflow

1. Keep the main branch deployable.
2. Create a separate branch for each feature or bug fix.
3. Open a pull request before merging changes.
4. Require automated tests to pass.
5. Ask at least one teammate to review changes where team size permits.
6. Use GitHub Issues to track tasks and defects.
7. Document architectural decisions in the repository.

A reasonable branch naming convention is:

```
feature/neighbourhood-scoring
feature/event-search
feature/food-search
fix/onemap-timeout
test/scoring-engine
```

Why this matters for GoWhere

The three features share routing, location services, and some data sources. Parallel development without clear interfaces could cause duplicated logic and merge conflicts.

I would assign ownership of individual modules while requiring the team to agree on shared interfaces early.

For example, the team should agree on the interface for route calculations before separate developers implement the event-search and food-search workflows.

That is more important than adopting a complicated branching strategy.

---

9. Deployment and scheduled tasks

Recommendation: start with a simple deployment

Use a managed Python hosting service or a small Linux server that supports the chosen web framework and persistent storage.

For a student project, avoid beginning with Kubernetes, multiple microservices, or a complex cloud architecture.

However, the deployment must support more than just serving web requests.

GoWhere needs:

- A persistent location for its database and validated data snapshots.
- Secure storage for API credentials.
- A way to run event ingestion periodically.
- A way to perform data refreshes and place-list builds.
- Logging and basic monitoring.
- A recovery procedure for failed updates.

The requirements specify that event ingestion runs every six hours when scheduling is enabled, while neighbourhood snapshots must be refreshed at least every three months. [cites: GoWhere_requirements_and_use_ca…]

A practical setup would be:

Task	Recommended mechanism
Serve web requests	Flask application process
Ingest events	Scheduled job every six hours
Remove ended events	Daily scheduled job
Refresh neighbourhood snapshot	Maintainer-triggered job at least every three months
Rebuild the place list	Maintainer-triggered job
Run automated tests	GitHub Actions
Store secrets	Hosting-platform secret manager or environment variables
Monitor failures	Application logs and basic health checks

Important deployment caveat: Do not rely on an in-memory session store if the application will run across multiple workers or instances. The class diagram describes per-session routing and search state held in memory. For a multi-worker deployment, use a shared session store or explicitly design a reliable

session-affinity strategy. A shared cache such as Redis may be appropriate if the deployment needs it, but it is unnecessary for the initial single-process setup.

---

10. Security and privacy

GoWhere's security requirements should influence the architecture from the start.

The application must not require account creation or collect personally identifiable information. User location, postal codes, comparison weights, and factor options must remain session-scoped and must not be written to URLs, cookies, logs, or disk. API credentials must remain server-side. [cites: GoWhere_requirements_and_use_ca…]

I recommend the following controls:

Risk	Recommended control
API credentials accidentally committed to Git	Environment variables, secret management, and a `.gitignore` file
Credentials exposed in browser requests	Make external API calls from the backend wherever appropriate
SQL injection	Parameterised SQL queries
Cross-site scripting	Jinja's contextual autoescaping and validation of untrusted input
Malicious instructions in event posts	Treat post content strictly as data; validate all extracted event fields
Accidental storage of private search inputs	Explicitly restrict what may be persisted and logged
Excessive API calls	Rate limiting, request caching, and configured daily caps
Corrupted data refresh	Validate new snapshots before activation
External service outage	Timeouts, error handling, and graceful fallbacks

One subtle point: the requirement says session-only inputs must not be written to cookies or disk. Therefore, the team should not assume that conventional cookie-based session storage is acceptable. Choose a session design that satisfies the actual privacy constraints.

Similarly, an environment variable is suitable for keeping a secret out of source control, but a managed secrets service may offer stronger operational controls for a production deployment.

---

11. What I would deliberately avoid

A beginner team can lose considerable time maintaining infrastructure that contributes little to the product's core functionality.

I would initially avoid the following:

Technology or approach	Why I would avoid it initially	When to reconsider
React or Next.js	Adds frontend framework and build complexity	Rich client-side interactions become a major requirement

Microservices	Introduces service communication, deployment, and monitoring overhead	Independent scaling or deployment becomes necessary
Kubernetes	Too much operational overhead for a small application	There is a demonstrated need for container orchestration
PostgreSQL plus PostGIS	More setup and maintenance than necessary for an initial SQLite deployment	Concurrent writes or advanced spatial queries justify the migration
Redis	Another service to deploy and maintain	Shared caching or session state becomes necessary
Celery and a message broker	More infrastructure than simple scheduled jobs initially require	Background workloads need queues, retries, and distributed workers
A full ORM migration	Could introduce unnecessary changes if the current data-access layer is already adequate	The team needs more sophisticated schema management or relational modelling

These are not inherently bad technologies. The recommendation is to introduce them in response to actual requirements rather than because they are common in larger production systems.

---

12. Implementation roadmap for the team

I would introduce the stack in the following order.

Phase 1 — Establish a working application

Technologies: Python, Flask, HTML, CSS, Jinja, Git, and GitHub.

Tasks:

- Set up the repository and development environment.
- Create the Flask application and shared page layout.
- Implement navigation between the three features.
- Establish naming conventions and module responsibilities.
- Configure basic logging and error handling.

Deliverable: A working web application skeleton.

Phase 2 — Implement the core business logic

Technologies: pandas, NumPy, SQLite, pytest.

Tasks:

- Implement the neighbourhood scoring engine.
- Implement event and food search logic.
- Define the data-access interfaces.
- Add input validation and deterministic sorting.
- Write unit tests for the calculation and filtering rules.

Deliverable: The core application logic works independently of the external services.

Phase 3 — Integrate external services

Technologies: `requests`, OneMap API, LTA DataMall, Telegram, Google Places, and an LLM API.

Tasks:

- Implement one adapter per external service.
- Configure credentials securely.
- Add timeouts, response validation, and fallback behaviour.
- Mock external APIs in tests.
- Implement route and location caching where appropriate.

Deliverable: All three features can use their required external data sources without depending directly on API implementation details.

Phase 4 — Complete the user experience

Technologies: JavaScript, Chart.js, CSS.

Tasks:

- Implement interactive search forms.
- Display neighbourhood scores and recommendations.
- Add maps, route details, and progress indicators.
- Implement filtering, sorting, and pagination.
- Test responsive layouts on desktop and mobile browsers.

Deliverable: Users can complete the main workflows from start to finish.

Phase 5 — Reliability and deployment

Technologies: pytest-cov, Ruff, GitHub Actions, scheduled jobs, and a suitable Python hosting service.

Tasks:

- Add integration and end-to-end tests.
- Test external-service failures and concurrent requests.
- Verify data-refresh recovery.
- Configure scheduled ingestion and maintenance tasks.
- Deploy the application and verify its privacy controls.

Deliverable: A maintainable deployment that meets the most important functional and non-functional requirements.

---

13. Final recommendation

If I were advising an entry-level team building GoWhere, I would settle on the following baseline:

Core stack

- Backend: Python + Flask
- Frontend: HTML + CSS + vanilla JavaScript + Jinja
- Data processing: pandas + NumPy
- Database: SQLite
- Visualisation: Chart.js
- Maps and routing: OneMap API
- External API calls: `requests`
- Testing: pytest
- Code quality: Ruff
- Version control: Git + GitHub
- CI: GitHub Actions
- Deployment: A managed Python host or small Linux server
- Scheduling: Cron or the hosting platform's scheduled-job facility

Three decisions matter more than adding more technologies:

1. **Maintain modularity.** Keep business logic separate from web routes, data access, and external-service adapters.
2. **Make failure handling a first-class requirement.** A failed API request or data refresh should not unnecessarily disable unrelated features.
3. **Automate testing and protect user data.** The scoring formulas, data-refresh process, external-service fallbacks, and session privacy constraints should all be verified systematically.

The proposed stack is deliberately conservative. It is capable of supporting GoWhere's three features while giving a beginner team a manageable codebase and a clear path to more sophisticated infrastructure if the project grows.

The priority should be to deliver a reliable, testable application—not to maximise the number of technologies in the stack.

Appendix B — Part 2 response (minimal skeleton), verbatim

Copied as ChatGPT displayed it. Its file-citation markers are shown as [cites: ...]: `GoWhere_requirements_and_use_ca…` *is the requirements document,* `class_diagram_mermaid` *the class diagram.*

I've created the **GoWhere Flask project skeleton** based on your Mermaid design class diagram. The structure follows the diagram's three main architectural layers: Boundary, Control, and Entity. [cites: class_diagram_mermaid]

1. Download the ZIP file

gowhere_flask_skeleton.zip📦 Download the GoWhere Flask Skeleton (sandbox:/mnt/data/gowhere_flask_skeleton.zip)

The ZIP contains 74 files, organized into a Python 3 and Flask project structure.

What I've included

- **Boundary layer:** Web routes and adapters for OneMap, DataMall, Google Places, Telegram, LLMs, data.gov.sg, and Overpass.
- **Control layer:** Search services, scoring engine and strategies, routing, location services, parking, and event ingestion.
- **Entity layer:** Events, places, neighbourhood snapshots, stores, opening hours, and session state.
- **Templates:** Placeholder pages for Live, Lepak, and Eat workflows, plus error pages.
- **Tests:** Placeholder test files for routes, search, scoring, adapters, routing, and stores.
- **Configuration:** Placeholder files for project dependencies, settings, environment variables, and the application entry point.

Your constraints are preserved: every file contains only TODO comments. There are no imports, class or function definitions, method signatures, variables, configuration values, or HTML tags.

---

2. Folder tree

```
gowhere_flask_skeleton/
├── README.md
├── pyproject.toml
├── requirements.txt
├── .env.example
├── .gitignore
├── run.py
│
├── config/
│   ├── __init__.py
│   └── settings.py
│
├── app/
│   ├── __init__.py
│   │
│   ├── boundary/
│   │   ├── __init__.py
│   │   ├── web_routes.py
│   │   ├── onemap_adapter.py
│   │   ├── token_source.py
│   │   ├── datamall_adapter.py
│   │   ├── llm_adapter.py
│   │   ├── telegram_post_source.py
│   │   ├── google_places_adapter.py
│   │   ├── daily_cap.py
│   │   ├── data_gov_adapter.py
│   │   └── overpass_adapter.py
│   │
│   ├── control/
│   │   ├── __init__.py
│   │   ├── lepak_search.py
│   │   ├── eat_search.py
│   │   ├── eat_ratings.py
│   │   ├── eat_hours.py
│   │   ├── scoring_engine.py
│   │   ├── scoring_strategy.py
│   │   ├── public_transport_scorer.py
│   │   ├── housing_affordability_scorer.py
│   │   ├── amenities_scorer.py
│   │   ├── greenery_scorer.py
│   │   ├── healthcare_scorer.py
│   │   ├── commute_scorer.py
│   │   ├── commute_router.py
│   │   ├── location_service.py
│   │   ├── event_router.py
│   │   ├── parking_service.py
│   │   └── ingestor.py
│   │
│   └── entity/
│       ├── __init__.py
│       ├── snapshot.py
│       ├── neighbourhood.py
│       ├── event_store.py
│       ├── event.py
│       ├── place_store.py
│       ├── place.py
│       ├── hours.py
│       ├── factor_choice.py
│       ├── session_store.py
│       └── session_state.py
│
├── templates/
│   ├── base.html
│   ├── live/
│   │   ├── submit.html
│   │   └── results.html
│   ├── lepak/
│   │   ├── submit.html
│   │   ├── results.html
```

```
│   │   └── event.html
│   ├── eat/
│   │   ├── submit.html
│   │   ├── results.html
│   │   ├── area.html
│   │   └── place.html
│   └── errors/
│       ├── 404.html
│       └── 500.html
│
├── static/
│   ├── css/
│   │   └── TODO.md
│   ├── js/
│   │   └── TODO.md
│   └── images/
│       └── TODO.md
│
└── tests/
    ├── __init__.py
    ├── conftest.py
    ├── test_web_routes.py
    ├── test_lepak_search.py
    ├── test_eat_search.py
    ├── test_eat_hours.py
    ├── test_scoring_engine.py
    ├── test_routing_services.py
    ├── test_stores.py
    └── test_adapters.py
```

---

3. Why each folder exists

Folder	Purpose
`config/`	Holds application configuration documentation and the planned settings structure, keeping configuration concerns separate from business logic.
`app/`	Contains the main application package and the components that implement the web application's architecture.
`app/boundary/`	Isolates the web interface and external-system integrations so the rest of the application does not need to handle API communication directly.
`app/control/`	Contains the application's workflows, business logic, search operations, scoring strategies, and routing services.
`app/entity/`	Organizes domain records, data-access components, stores, and session-state representations.
`templates/`	Holds the HTML template files that Flask will eventually render for the application's pages.
`templates/live/`	Groups the pages associated with the Live workflow.
`templates/lepak/`	Groups the event-search, event-results, and event-detail pages.
`templates/eat/`	Groups the food-search, browsing, results, and place-detail pages.
`templates/errors/`	Reserves templates for common HTTP error pages, such as 404 and 500.
`static/`	Reserves space for frontend assets served directly by Flask.
`static/css/`	Will hold the application's stylesheets.
`static/js/`	Will hold browser-side JavaScript.

`static/images/`	Will hold image assets used by the interface.
`tests/`	Separates automated tests from application code so individual components and their interactions can be verified.

Root-level files

These files are not folders, but they have distinct responsibilities:

File	Purpose
`README.md`	Documents the project's purpose, architecture, setup, and development workflow.
`pyproject.toml`	Reserves space for Python project metadata and development-tool configuration.
`requirements.txt`	Documents where runtime dependencies, including Flask, will eventually be listed.
`.env.example`	Documents the environment variables the application will eventually require without storing real secrets.
`.gitignore`	Documents which local files and generated artifacts should be excluded from version control.
`run.py`	Reserves the entry point for starting the application during development.
`app/__init__.py`	Documents the intended Flask application setup and application-factory responsibilities.
`tests/conftest.py`	Reserves space for shared pytest fixtures and test setup.

---

4. Architectural notes

I made three deliberate decisions when translating your diagram into a project structure.

1. Preserve the three-layer architecture.

The Boundary, Control, and Entity packages reflect the diagram's organization rather than placing all classes into a single application module. This should make the project easier to navigate as it grows.

2. Give the scoring strategies separate files.

The six scoring strategies each have their own placeholder file, alongside the shared strategy and scoring-engine files. This mirrors the Strategy pattern shown in the diagram and keeps individual scoring concerns separate.

3. Keep external integrations at the boundary.

Adapters for OneMap, DataMall, Google Places, LLMs, and other external services live under `app/boundary/`. Their consumers and business workflows are organized separately under `app/control/`.

One important limitation: this is a structural skeleton, not an executable Flask application yet. Since you requested comment-only files, Flask cannot run until the relevant TODOs are implemented.

Recommended next step: implement the application factory and basic routing first, then add the Entity layer, Control services, and external adapters incrementally. This will let you establish a working Flask application before implementing the more complex business logic.